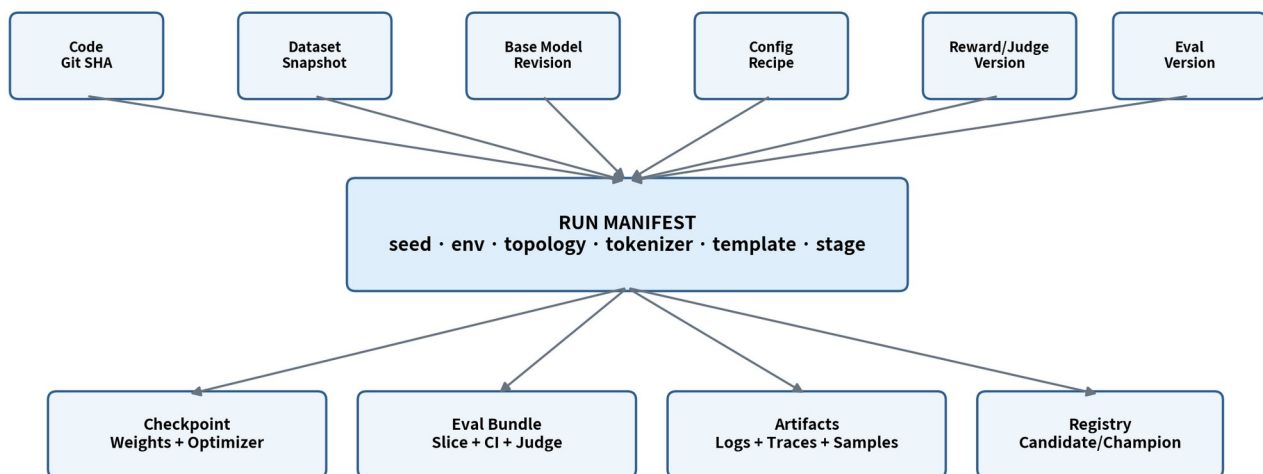


LLM 后训练之 Experimentation & Governance

实验体系、版本治理与可复现性

深度研究与工程手册



可复现的对象不是“checkpoint”，而是完整 experiment lineage DAG

核心问题：如何让每一次 SFT / DPO / RLVR 实验都能被解释、比较、复现、审计和安全晋级，而不是变成“某个 checkpoint 看起来更好”？

研究检索截至 2026-08-13 · 聚焦 Post-Training Experiment Design · Reproducibility · Lineage · Registry · Release Governance

基于公开论文/官方文档：Ai2 Tulu 3、PyTorch、Hugging Face、MLflow、Weights & Biases、Delta Lake、DVC。

执行摘要

当后训练从一次 SFT 扩展成 Prompt/Data → SFT → Preference → RLVR → Distillation → Eval 的多阶段系统后，最稀缺的并不是 GPU，而是“可信比较”。如果一个模型提高了 1.2 分，你必须知道：到底是数据、sampling、reward、judge、模板、seed、checkpoint、eval revision，还是随机波动造成的。Tulu 3 的公开实践把完整数据、训练代码、decontamination、evaluation 和 recipe 一起发布，本质上说明现代 post-training 的可复现对象是整个 pipeline，而非单独权重。[1]

第一性结论：Experiment Governance 的目标不是“多记录日志”，而是建立从 Hypothesis → Run → Evidence → Decision → Release 的可审计因果链。

层	核心问题	最小产物
Hypothesis	这次只想验证什么？	experiment hypothesis / expected direction
Run Identity	这次训练到底是什么？	immutable Run Manifest
Lineage	输入和输出来自哪里？	data/model/reward/eval DAG
Evidence	差异是否超过噪声？	paired eval + seed/CI
Attribution	是哪一个变量导致变化？	ablation / controlled comparison
Governance	是否允许晋级？	release gates + registry
Rollback	出现回归能否恢复？	candidate/champion + immutable artifacts

本手册将“可复现性”分成三层：bitwise/exact、statistical、behavioral。PyTorch 官方明确指出，跨 PyTorch 版本、跨平台、甚至 CPU 与 GPU 之间都不能保证完全复现；deterministic algorithms 也通常以性能为代价。[2] 因而大型 LLM 后训练不应把“同 seed”当作可复现性的充分条件。

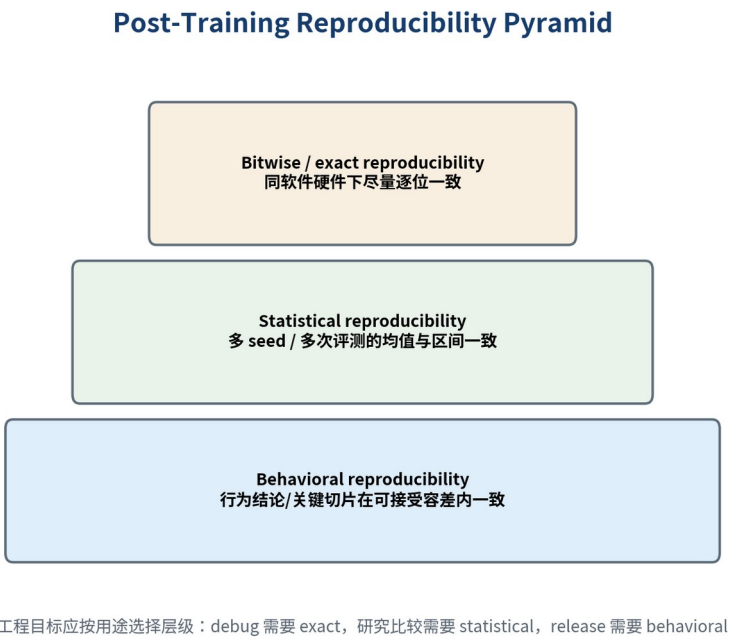


图 1：后训练的三层可复现性。

1. 从 Experiment Tracking 升级为 Experiment Governance

传统 Experiment Tracking 记录 params / metrics / artifacts；MLflow Tracking 也以 Run 为核心，记录参数、代码版本、指标和输出 artifacts，并支持把 dataset 作为输入记录到 run。[4][5] 但 Post-Training Governance 需要再向前一步：不仅“记录发生了什么”，还要定义哪些变量允许变化、哪些 artifact 必须冻结、哪些证据满足 release gate。

Tracking	Governance
记录 lr、loss、GPU hours	声明实验假设和唯一自变量
保存 checkpoint	绑定 data/base/reward/eval 版本
画 reward curve	判断 reward 变化是否转化为独立 eval 改善
保存模型	注册 candidate/champion，支持 rollback
比较多个 run	用 paired design / slice / CI 做可信比较

推荐规则：任何不能回答“相对于哪个 baseline、只改变了什么、证据是什么”的 run，都不应被称为有效实验。

2. Run Manifest：后训练实验的最小原子

一个生产级 run_id 应该只是一把索引键；真正的语义放在不可变 manifest 中。至少需要冻结六类版本：Data、Model、Reward/Judge、Eval、Code、Environment。

Six-version rule: 所有核心对象都必须有不可变版本



只记录 model_version 远远不够；reward/eval/data version 漂移会让“同一个实验”失去语义

图 2: Six-version rule。

字段	示例	缺失后的典型后果
code_git_sha	6f3a...	同 config 但实现已变化
dataset_digest	sft_mix@sha256:...	样本增删/过滤无法追踪
base_model_revision	model@commit	“同名 base model” 实际不同
reward/judge_version	rm-v12 / judge-prompt-v5	reward 漂移伪装成 policy 改善
eval_revision	eval-suite@2026-08-01	benchmark template/judge 改变
environment	torch/cuda/nccl/container digest	数值与性能差异无法解释
tokenizer/template	tokenizer SHA + chat_template SHA	序列化目标已改变
seed / data_seed	42 / 1701	sampling 与 initialization 噪声不可分离
distributed topology	TP/PP/DP/FSDP	数值与吞吐不可直接对比

```
run_id: pt-sft-20260813-017
stage: sft
hypothesis: "加入高难 tool-use slice 可提升 tool_success, 且 general eval 下降 < 0.3"
code:
  git_sha: 6f3a91d
  container_digest: sha256:...
model:
  base_repo: org/base-model
  base_revision: 3c27...
data:
  train_snapshot: sft_mix_v18@sha256:...
  sampler_manifest: sampler_v7@sha256:...
  tokenizer_revision: tok@41ab...
  chat_template_revision: tpl@8e01...
optimization:
  seed: 42
```

```
data_seed: 1701
lr: 1.0e-5
global_batch_tokens: 4194304
eval:
  suite_revision: eval_2026_08_01
  judge_revision: judge_v5
release:
  baseline_run: pt-sft-20260810-009
  gates: [tool_success, general_regression, safety_regression, cost]
```

3. Data Versioning: 不要只版本化“文件”，要版本化“训练经验”

MLflow Dataset Tracking 为 dataset 保存 name、digest、source、schema/profile，并将其绑定到 run；Hugging Face 的 model/dataset repositories 是 Git repositories，可通过 commit/tag 固定 revision；Delta Lake 的 time travel 允许按 table version 访问历史快照。[5][8][10] 对 Post-Training 来说，还需要额外版本化 filtering、dedup、mixing、sampling 和 pair/trajectory construction。

对象	应该冻结什么
Raw pool	source revision / license / ingest time
Filtered pool	filter code + filter thresholds + rejection stats
Dedup	embedding/model + threshold + clustering algorithm
Mixture	每 dataset 的 sample/token weight
Sampler	temperature/cap/curriculum/random seed
Preference pair	candidate generator + judge + pair selection rule
RL experience	policy_version + rollout config + verifier_version
Eval split	exact sample IDs + template + grader

Know-how：dataset digest 不能只 hash 文件路径。对 streaming / database / lakehouse 数据，必须绑定 snapshot/version；对合成数据还要绑定 generator、prompt template、sampling config 与 verifier。否则“重新生成同一个 dataset”并不等于同一训练输入。

4. Model / Checkpoint Lineage: Checkpoint 是状态，不是结论

MLflow Model Registry 把 model version 与产生它的 experiment/run 关联，并支持 version、tag、alias 等治理机制；例如 champion/challenger alias 可将部署语义与具体版本解耦。[6] Hugging Face Model Cards 也建议记录训练参数、实验信息、训练数据和 evaluation results。[9]

Checkpoint Artifact	Release Model
用于恢复训练	用于跨团队消费/部署

包含 optimizer / scheduler / RNG state	至少包含 weights/tokenizer/config/template
可以高频产生	应经过独立 eval gate
由 global_step 标识	由 model version + lineage 标识
可能只是中间态	必须有 intended use / limitations / eval report

不要把“latest checkpoint”当成“best model”。生产 registry 中 latest、best-on-dev、candidate、champion 应是不同概念。

5. Reward / Judge / Eval 也必须版本化

后训练最隐蔽的实验污染之一，是只冻结 policy checkpoint，却让 Reward Model、LLM Judge prompt、verifier code 或 eval template 发生变化。此时 reward 曲线和 benchmark 分数都无法与旧 run 直接比较。Tulu 3 明确把 standardized evaluation suite、decontamination 与 development/unseen eval 一起作为 recipe 的组成部分，并强调 eval 要 reproducible、评测 unseen task generalization。[1]

组件	版本对象	必须记录
Reward Model	weights + tokenizer	RM checkpoint / calibration set
LLM Judge	model + system/rubric prompt	API/model revision + prompt SHA + decoding
Rule Verifier	code + test data	git SHA + test version
Environment	container + initial state	image digest + task seed
Eval Benchmark	sample IDs + prompt template	dataset revision + template SHA
Aggregation	metric implementation	metric version + bootstrap config

工程原则：训练用 reward 与独立 release eval 必须保持组织与版本隔离。否则 reward hacking 会被同源 eval 误认为能力提升。

6. 随机性：Seed 只是入口，不是答案

PyTorch 官方文档明确指出，完全可复现结果不保证跨 release、commit 或 platform；即使 seed 一致，CPU/GPU 也可能不同。官方建议控制 PyTorch/Python/NumPy RNG、DataLoader worker seeds，并可用 torch.use_deterministic_algorithms() 强制确定性算法，但会损失性能。[2] Hugging Face Trainer 进一步区分 seed 与 data_seed，并提供 full_determinism，官方同样提醒其会降低性能。[3]

随机源	LLM 后训练中的表现	治理方式
Parameter/init RNG	adapter/head/random init	记录 model_init + seed
Data order	shuffle / distributed sampler	独立 data_seed + epoch cursor
Generation RNG	temperature / multinomial	rollout seed + sampling config
Distributed collectives	归约次序/数值误差	记录 topology/software/hardware
Kernel nondeterminism	flash/attention/atomic ops	debug run 开 determinism
External Judge/API	服务端模型更新	pin revision；必要时保存原始 judge outputs

建议：Debug 用 exact mode；算法比较用 multi-seed statistical mode；Release 用 behavioral regression mode。不要为了 bitwise determinism 牺牲所有生产吞吐。

7. 三层可复现性：Exact / Statistical / Behavioral

层级	目标	典型用途	通过标准
L1 Exact	相同环境下尽量逐位一致	debug / checkpoint resume	loss/weights/state 一致
L2 Statistical	随机 run 的分布结论一致	算法/数据 ablation	均值方向一致、CI 可接受
L3 Behavioral	关键产品行为稳定复现	release / rollback	核心 slice 不回归，风险指标受控

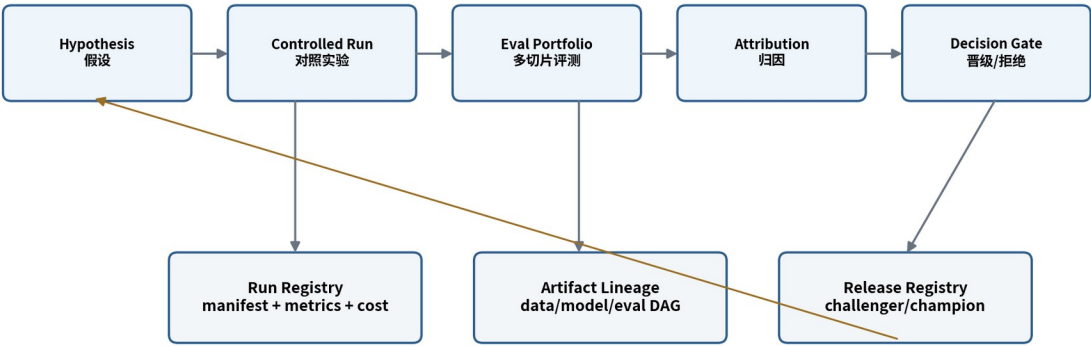
这里的三层框架是工程抽象：它接受 PyTorch 所说明的“跨环境完全确定性不可保证”的事实，同时把不同目标所需的复现强度区分开。对几十/几百 GPU 的 RLVR，如果要求每个 kernel 都 exact，成本通常不合理；但对 reward parser bug，又需要可以精确重放同一 trajectory。

8. Stage-specific Experiment Unit：不同阶段不能共用同一比较逻辑

阶段	主要自变量	最重要的噪声/混淆	推荐最小实验单位
SFT	data mix / lr / epochs / mask	data order、template、packing	固定 data snapshot 的 paired runs
DPO	pair mix / beta / loss	pair generator、length bias	同 pairs + 同 ref model 的对照
RM/PRM	label mix / model size	judge noise、class balance	held-out preference / calibration
RLVR	prompt mix / G / reward / clip	rollout RNG、policy freshness	group-level stats + multi-window eval
Distill	teacher/data/loss	teacher quality/capacity gap	固定 teacher snapshot / student seeds
Agent RL	environment/reward/tool	environment nondeterminism	trajectory replay + task-state verification

Tulu 3 的公开开发过程包含多轮 data mixture ablation 与 hyperparameter tuning，并报告一些方法在调参后表现相近，因此选择更简单高效的 DPO。这是一个重要实验治理信号：不仅报告 winner，也要记录“哪些复杂方法没有稳定收益”。[1]

9. Hypothesis-driven Ablation：一次只回答一个问题



Experimentation = Hypothesis-driven search + controlled evidence + auditable decision

图 3：从 hypothesis 到 release 的实验闭环。

最常见的坏实验是同时改变 data mix、learning rate、template、judge 和 eval，然后看到分数上涨就归因给其中一个因素。对于昂贵 LLM 后训练，无法做全因子实验时，至少需要明确“主要处理变量”和“保持不变的控制变量”。

实验类型	适用问题	优点	风险
One-factor ablation	某个具体组件是否有效	解释简单	忽略交互作用
2×2 factorial	两个因素是否交互	可估计 interaction	成本增加
Sequential tuning	先 data 后 optimizer	符合工程节奏	顺序造成路径依赖
Matched-budget	方法 A/B 在同 token/GPU budget	公平	可能压制需要不同 budget 的算法
Matched-performance	达到同指标所需成本	产品视角好	需要更多运行

每个实验卡至少写：Hypothesis、Treatment、Control、Primary Metric、Guardrails、Compute Budget、Decision Rule。

10. Paired Evaluation 与统计置信度

后训练评测经常是同一批 prompts 上比较两个 checkpoint，因此优先使用 paired design：对每个样本计算 $\Delta_i = \text{score}(\text{candidate}_i) - \text{score}(\text{baseline}_i)$ ，再对 Δ 的均值/胜率进行 bootstrap 或合适的成对统计。相比只比较两个独立平均数，它能抵消任务难度差异。

指标类型	推荐比较方式
Exact/Pass Fail	paired win/loss; bootstrap CI; McNemar (适合二元配对)

Scalar score	paired delta + bootstrap CI
LLM preference	swap order + pairwise win rate + CI
Agent task	task-level paired success + cost/steps delta
RL reward	仅作训练健康指标，不作为 release 结论

Know-how：不要把 0.3 分提升自动当成 improvement。首先估计该 eval 在重复采样、judge noise、prompt template 下的方差；然后定义 Minimum Detectable Effect / practical significance。统计显著但产品无意义的变化同样不值得晋级。

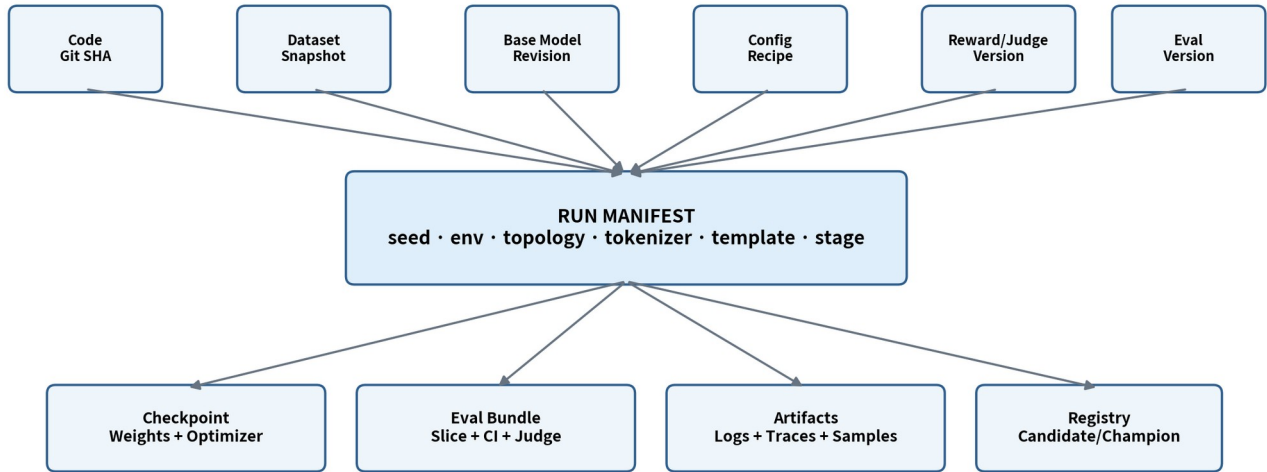
11. Slice-first：平均分是最后看的指标之一

Post-training 通常是多目标优化。一个总平均分可以掩盖“coding +3、instruction following -4、安全 slice -2”。Tulu 3 的开发方式就是先构建 skill-specific mixtures 和模型，观察单项上限，再迭代 add/remove datasets 改善落后 skill。[1]

Slice 维度	例子
Capability	reasoning / code / tool / long-context / writing
Difficulty	easy / learnable frontier / hard / adversarial
Source	real user / synthetic / benchmark / failure-mined
Length	short / medium / long CoT
Language	zh / en / multilingual
Risk	normal / sensitive / refusal / jailbreak
Agent state	first try / retry / recovery / tool-error
Cost	tokens / tool calls / wall-clock

Release decision 应该基于 Metric × Slice × Model Version 三维矩阵，而不是 leaderboard 单列。

12. Experiment Lineage DAG：从表格升级为图



可复现的对象不是“checkpoint”，而是完整 experiment lineage DAG

图 4：推荐的实验 lineage DAG。

W&B Artifacts 把 runs 与 artifact inputs/outputs 表示为有向无环图，并强调 lineage 对 reproducibility、version control、auditing 和 collaboration 的价值。[7] 对后训练平台，可以把同一思想扩展到 data snapshot、pair dataset、reward model、policy checkpoint、eval bundle 与 release candidate。

推荐边类型：USED_DATA、INITIALIZED_FROM、GENERATED_BY、FILTERED_BY、JUDGED_BY、TRAINED_FROM、EVALUATED_WITH、PROMOTED_TO。这样当一个 release 出问题，可以沿 DAG 向上追溯到准确的数据与 judge 版本，而不是人工翻日志。

13. Experiment Registry vs Model Registry vs Dataset Registry

Registry	保存什么	核心查询
Experiment Registry	run manifest / metrics / artifacts / hypothesis	“哪些 run 只改了 beta？”
Dataset Registry	snapshot / digest / schema / lineage / license	“这个 checkpoint 用过哪些样本？”
Model Registry	model versions / aliases / gates / model card	“当前 champion 来自哪个 run？”
Eval Registry	eval revisions / templates / judge / slices	“v4 和 v5 benchmark 有何不同？”
Reward Registry	RM/verifier/judge revisions	“reward 曲线变化是否来自 reward 更新？”

MLflow Tracking、Dataset Tracking 与 Model Registry 已提供这些概念的一部分：run 记录 params/metrics/artifacts，dataset 有 digest/source，registered model version 与 run 建立 lineage，并可通过 aliases 管理 champion 等部署语义。[4][5][6]

14. Champion / Challenger：晋级必须显式化

MLflow Model Registry 的 alias 机制允许用 champion 等命名引用具体 model version，并可重定向到新版本；这提供了一个实用的 release governance 抽象。[6] 对 LLM 后训练，应进一步把 promotion 绑定到 gate report，而不是手工改 alias。

Gate	示例
Primary capability	目标能力 $\geq$ baseline + MDE
General regression	核心通用 slice 不下降超过阈值
Safety / policy	高风险 slice 不恶化
Latency/cost	token / tool / latency 成本在预算内
Judge robustness	swap/order/alternate judge 结论一致
Reproducibility	第二 seed / rerun 方向一致
Artifact completeness	manifest/card/lineage/weights 全部存在

自动晋级应 fail-closed：任何 gate 缺数据，都应该是“未通过”，而不是默认通过。

15. Rollback 与 Incident Reproduction

真正成熟的治理不是“可以找回旧权重”，而是可以重放产生该行为的完整环境：model revision、tokenizer/chat template、system prompt、tool schema、judge/reward、inference config、environment state。Hugging Face repos 通过 Git commit/tag 提供模型和数据 revision；Delta Lake time travel 也展示了对数据快照进行历史访问的模式。[8][10]

- 保留 release candidate 的 immutable inference bundle，而不是只保留 safetensors。
- 所有线上 incident 样本保存 request/response/tool trace 的可重放版本，并脱敏。
- rollback alias 与重新训练是两件事：前者分钟级恢复，后者用于根因修复。
- 当问题来自 judge/eval/harness，而非 policy 时，不要无意义地回滚模型。

16. Experiment Cost Accounting：比较效果必须同时比较代价

后训练实验经常因为不同方法使用不同 rollout N、sequence length、judge calls 或 tool/environment cost 而出现“隐形预算不公平”。因此 experiment manifest 应同时记录训练 tokens、generated tokens、GPU-hours、judge calls、tool calls、wall-clock、storage 与失败重试成本。

成本单位	适合比较什么
GPU-hour	总体硬件消耗
Train token	SFT/DPO 优化规模
Generated token	RL/BoN/teacher synthesis 搜索成本

Verifier/Judge call	偏好/奖励成本
Environment step	Agent RL 的真实交互成本
Accepted useful sample	Data Factory 的有效产出
Δ Eval / 1000 GPU-hours	效果/成本联合指标

Know-how：做 matched-compute ablation 时，不要只匹配 train FLOPs；RLVR 的 rollout 与 verification 可能远高于 optimizer update 的成本。

17. Hyperparameter Search：搜索空间本身也要版本化

学习率、batch、beta、clip、KL、group size、temperature、max_tokens、data mixture 等变量存在强交互。DVC 的 experiment versioning 思想把 metrics、hyperparameters 与 artifact information 作为可版本化实验状态，强调 experiment tracking 与 version control 的结合。[11]

搜索策略	适合
Manual hypothesis sweep	少量高成本 LLM 实验
Grid	低维离散空间
Random	高维、先找重要变量
Bayesian/SMBO	单次 run 昂贵但反馈相对稳定
Successive Halving	可用早期 proxy 淘汰
Population/online tuning	长 RL run，但治理和因果解释更复杂

搜索实验必须保存 `search_space_version + scheduler_version`；否则“best hyperparameter”无法在后续复现。

18. Environment & Dependency Governance

软件环境是模型语义的一部分。PyTorch 已明确说明不同 release/platform 可能无法完全复现。[2] 对 LLM post-training 还要冻结 CUDA、cuDNN、NCCL、attention backend、quantization kernel、compiler flags，以及生成引擎版本。

必须记录	为什么
container digest	requirements.txt 不能完全描述二进制环境
CUDA/cuDNN/NCCL	collective 与 kernel 数值/性能变化
GPU model / topology	吞吐与数值路径可能不同
attention backend	Flash/SDPA backend 可能改变数值与 determinism
vLLM/SGLang revision	rollout sampling semantics 可能变化
tokenizer lib revision	special token/template 处理可能变化

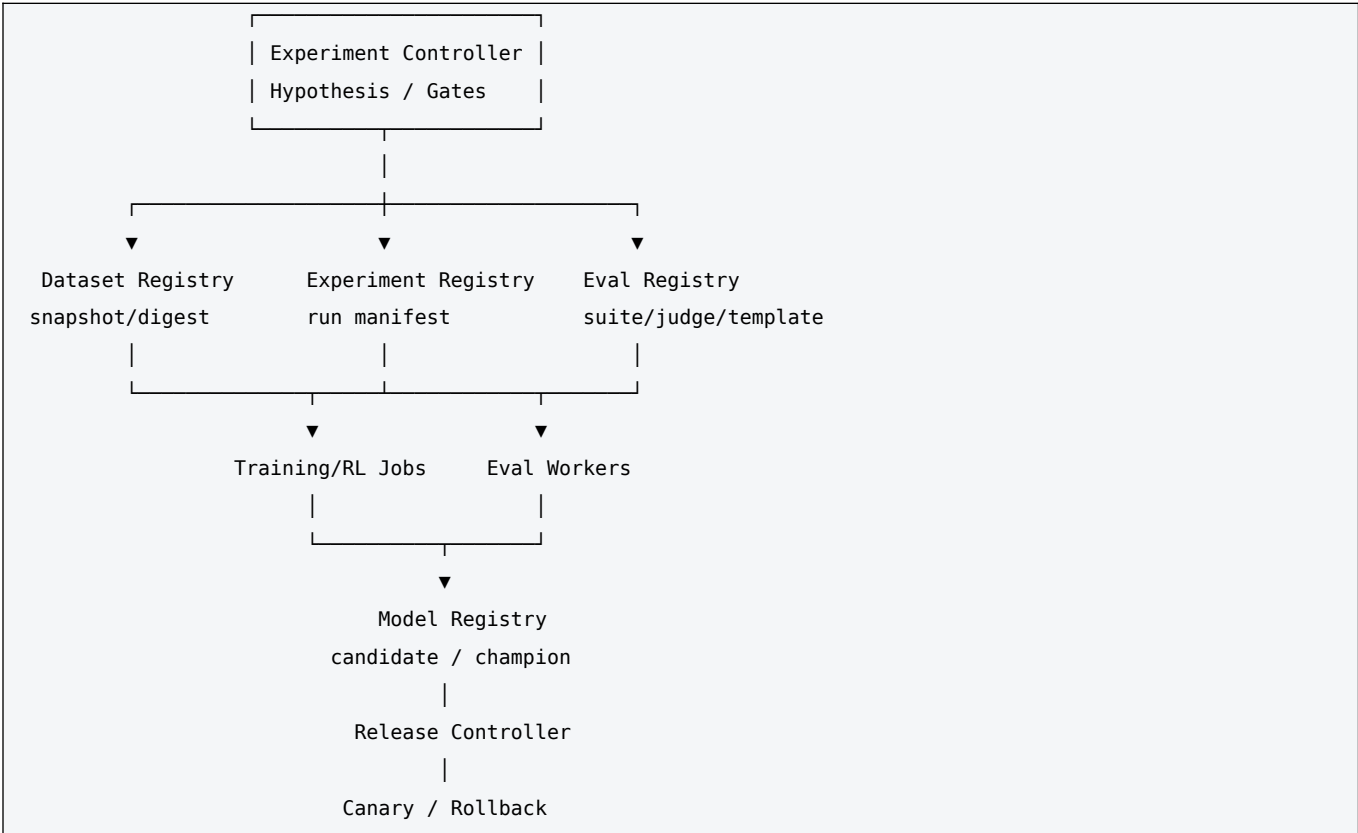
19. Governance 不只是复现：还包括 Provenance、License、PII 与访问控制

Hugging Face Dataset Cards 建议记录 dataset 的内容、license、语言、大小与潜在 bias；Model Cards 则建议记录 intended uses、limitations、训练参数、datasets 与 evaluation results。[12][9] 对企业后训练，需要进一步把这些信息变成机器可检查的 policy gate。

治理对象	建议自动检查
Data provenance	source、license、consent/usage policy
PII	检测、脱敏、访问权限、保留期限
Synthetic data	generator/judge provenance
Eval leakage	train/eval overlap + decontamination status
Model release	intended use / limitation / risk report
Secrets	run logs/tool traces 中凭据扫描
Access	谁可读 raw logs、谁可 promote champion

20. Production Reference Architecture

建议把 Experimentation & Governance 作为 Infrastructure 上层的 Control Plane，而不是由训练脚本临时写日志。



21. 18 类常见 Failure Modes

#	Failure	表现 / 后果
1	Only checkpoint version	无法知道 data/reward/eval 版本
2	Mutable dataset path	相同 run config 得到不同数据
3	Same seed fallacy	误以为 seed 相同就完全可复现
4	Eval revision drift	“提升”来自 template/judge 更新
5	Reward drift	reward 曲线跨版本不可比
6	Multiple variables changed	无法归因
7	No baseline link	run 没有明确对照组
8	Single seed winner	选中了随机幸运 run
9	Average-only decision	关键 slice 回归被平均值掩盖
10	Latest=best	错误发布中间 checkpoint
11	No compute normalization	方法靠更多 rollout 赢
12	No negative-results log	团队重复踩同一失败路线
13	No environment digest	依赖升级后无法复现
14	No generator provenance	synthetic data 无法审计
15	No release gate	人工凭感觉 promote
16	No rollback bundle	只能找旧权重，无法重放行为
17	Judge not pinned	外部 judge 更新导致历史分数失效
18	Governance bypass	紧急实验直接进入 production

22. 成熟度模型：L0 → L4

Level	系统状态	关键能力
L0 Ad-hoc	目录名 + 手工表格	无法可靠复现
L1 Tracked	params/metrics/checkpoints	基本 run tracking
L2 Versioned	code/data/model/eval 版本化	可重复与可回滚
L3 Governed	hypothesis + gates + registry + lineage	可信比较与受控发布
L4 Adaptive	自动 gap/experiment scheduler + causal evidence	持续 post-training control plane

L4 的目标不是“自动跑更多实验”，而是自动识别最有价值的不确定性，并用最小计算预算消除它。

23. 90 天落地路线

阶段	目标	交付物
Day 1-15	统一 Run Manifest	run schema + code/data/model revision
Day 16-30	Dataset/Eval Registry	digest、snapshot、eval revision

Day 31-45	Model Registry + Gates	candidate/champion + rollback
Day 46-60	Paired Eval / CI	slice dashboard + regression bank
Day 61-75	Lineage DAG	artifact input/output graph
Day 76-90	Experiment Controller	hypothesis → run → evidence → promote

24. 可直接复用的 Governance Contract YAML

```
experiment:
  id: pt-rlvr-20260813-042
  owner: reasoning-team
  hypothesis: >-
    Dynamic sampling improves informative-group ratio by >= 8%
    and MATH slice by >= 1.0 without >0.3 regression on general eval.
  baseline_run: pt-rlvr-20260810-031
  treatment:
    dynamic_sampling: true
  controls:
    prompt_snapshot: rlvr_prompts_v12@sha256:...
    base_checkpoint: dpo-v19@sha256:...
    verifier_version: math-verifier-v7
    eval_revision: posttrain-eval-2026-08-01
  reproducibility:
    code_sha: 8b4c...
    container_digest: sha256:...
    seed: 42
    rollout_seed: 31415
    data_seed: 1701
    topology: {tp: 4, pp: 1, dp: 32}
  evidence:
    primary_metric: math_pass1
    paired_eval: true
    bootstrap_samples: 10000
    required_seeds: 2
  guardrails:
    general_regression_max: 0.3
    safety_regression_max: 0.0
    gpu_hours_max: 12000
  promotion:
    require_all_gates: true
    alias_on_pass: challenger
```

25. Know-Why / Know-How 总结

主题	Know-Why	Know-How
Run Manifest	checkpoint 不能描述完整实验	冻结六版本 + seed/topology/template
Reproducibility	exact 并非总能/总应实现	按 exact/statistical/behavioral 分层
Lineage	实验是 DAG，不是表格行	显式 input/output artifact edges
Ablation	LLM run 太贵，更要保证可归因	hypothesis + treatment/control
Evaluation	单平均分会掩盖回归	paired + slice + CI + guardrail
Registry	latest 不等于 best	candidate/champion aliases + gates
Rollback	旧权重不足以复现 incident	保存完整 inference/eval bundle
Governance	数据/模型不是唯一版本对象	reward/eval/code/env 同样版本化
Cost	性能可通过额外 search 买到	matched-budget / Δ Eval per GPU-hour

最终心智模型：Post-Training Experimentation 的产物不是“一个更高分的 checkpoint”，而是一条可信的证据链：Why → What Changed → What Stayed Fixed → What Improved → What Regressed → How Certain → How Much Cost → Can We Reproduce → Can We Roll Back。

参考资料与推荐阅读顺序

[1] **Ai2. Tulu 3: The next era in open post-training. 2024.**
<https://allenai.org/blog/tulu-3-technical>

[2] **PyTorch. Reproducibility. Documentation, updated 2026.**
<https://docs.pytorch.org/docs/stable/notes/randomness.html>

[3] **Hugging Face Transformers. Trainer - Reproducibility / seed / data_seed / full_determinism.**
https://huggingface.co/docs/transformers/en/main_classes/trainer

[4] **MLflow. ML Experiment Tracking.**
<https://mlflow.org/docs/latest/ml/tracking/>

[5] **MLflow. Dataset Tracking.**
<https://mlflow.org/docs/latest/dataset/>

[6] **MLflow. Model Registry - lineage, versions, aliases, tags.**
<https://mlflow.org/docs/latest/ml/model-registry/>

[7] **Weights & Biases. Artifact lineage graphs.**
<https://docs.wandb.ai/models/artifacts/explore-and-traverse-an-artifact-graph>

[8] **Hugging Face Hub. Repositories / branches / tags / revisions.**
<https://huggingface.co/docs/hub/main/repositories>

[9] **Hugging Face Hub. Model Cards.**
<https://huggingface.co/docs/hub/en/model-cards>

[10] **Delta Lake. Time travel / versioned data snapshots.**
<https://docs.delta.io/>

[11] **DVC. ML Experiment Versioning.**
<https://dvc.org/blog/ml-experiment-versioning/>

[12] **Hugging Face Hub. Dataset Cards.**
<https://huggingface.co/docs/hub/datasets-cards>

[13] Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training.

<https://arxiv.org/abs/2411.15124>

建议阅读顺序：先读 Tulu 3 理解“完整 recipe 的可复现对象”，再读 PyTorch/Trainer 理解 determinism 的边界，然后用 MLflow/W&B/HF/Delta/DVC 的官方概念构建 Experiment Registry、Lineage、Versioning 与 Release Governance。